

Uppsala University
Department of Information Technology

Assignment 4

Large Scale Machine Learning

Course: Data Engineering II

Arnab Kumar Ghosh

May 11, 2026

Contents

1 Task 1: Distributed Machine Learning (Mandatory)	2
1.1 Task 1.1: Hyperparameter Tuning	2
1.1.1 Problem Setup with scikit-learn	2
1.1.2 Distributed Hyperparameter Tuning with Ray Tune	3
1.1.3 Results	5
1.2 Task 1.2: Distributed Training of Deep Neural Networks	6
1.2.1 TensorFlow Distributed Training Tutorial	6
1.2.2 Data Parallelism vs. Model Parallelism	7
2 Task 2: Federated Learning (Extra Credit)	7
2.1 Simulation Setup	7
2.2 Federated Training	8
2.3 Analysis	9
2.3.1 How PySyft's <code>PointerTensor</code> / <code>ActionObject</code> Prevents Raw Data Access	9
2.3.2 Federated Model Accuracy vs. Single-Worker Baseline	9
2.4 Written Reflection	10
2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity	10

1 Task 1: Distributed Machine Learning (Mandatory)

1.1 Task 1.1: Hyperparameter Tuning

1.1.1 Problem Setup with scikit-learn

The cover-type prediction problem was set up using the `fetch_covtype` dataset from scikit-learn, which contains 581 012 samples and 54 features describing cartographic variables of forest patches in Colorado, with the goal of classifying seven forest cover types.

A `RandomForestClassifier` was instantiated with its default parameters. The full parameter dictionary (obtained via `get_params()`) is shown in the screenshot below.



Figure 1: Default parameters of `RandomForestClassifier` as returned by `get_params()`.

The model was trained on 80 % of the data (stratified split) and evaluated on the remaining 20 % with 3-fold cross-validation on the training set. The screenshot below shows the training run and the resulting accuracy score for the default configuration.



Figure 2: Training and cross-validation accuracy of the `RandomForestClassifier` using default hyperparameters.

1.1.2 Distributed Hyperparameter Tuning with Ray Tune

A grid search over the following hyperparameter space was performed using **Ray Tune** with a `GridSearchCV` strategy:

- `max_depth`: {10, 20, 30, None}
- `n_estimators`: {50, 100, 200}
- `ccp_alpha`: {0.0, 0.001, 0.01}

This yields $4 \times 3 \times 3 = 36$ candidate combinations, each evaluated with 3-fold cross-validation (108 total fits). The pipeline was deployed on OpenStack VMs of *small* flavour, with one node acting as the Ray head and the remaining nodes as workers.

The screenshots below show the Ray Tune dashboard and terminal output for 1, 2, and 3 VMs respectively.



Figure 3: Ray Tune grid search running on **1 VM** (head node only).

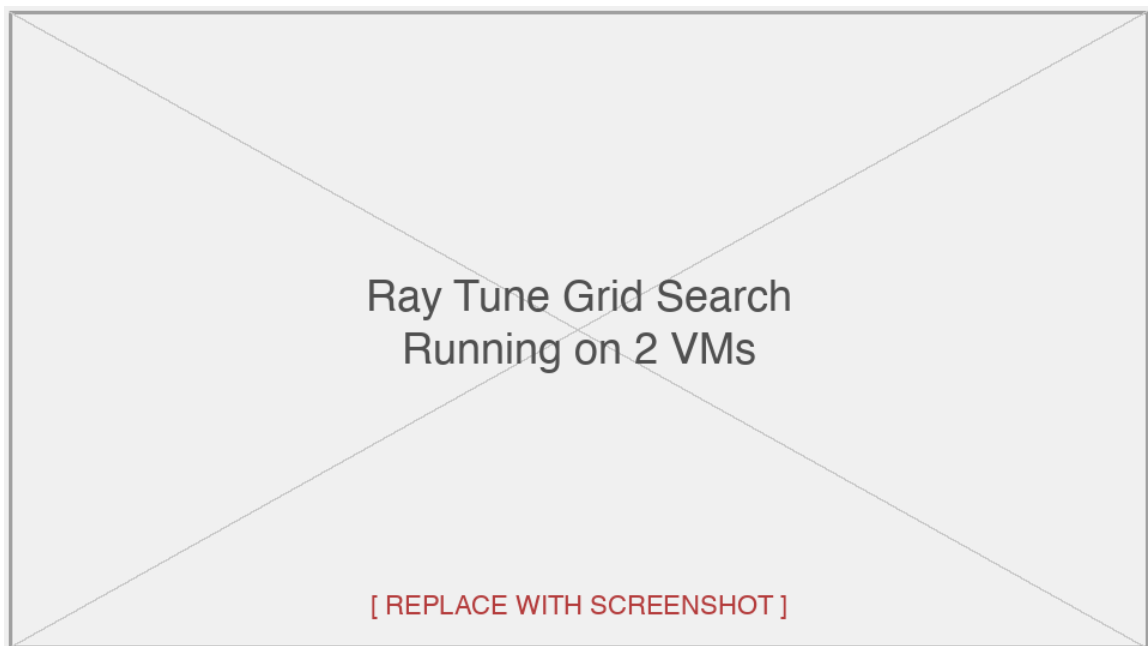


Figure 4: Ray Tune grid search running on **2 VMs** (1 head + 1 worker).

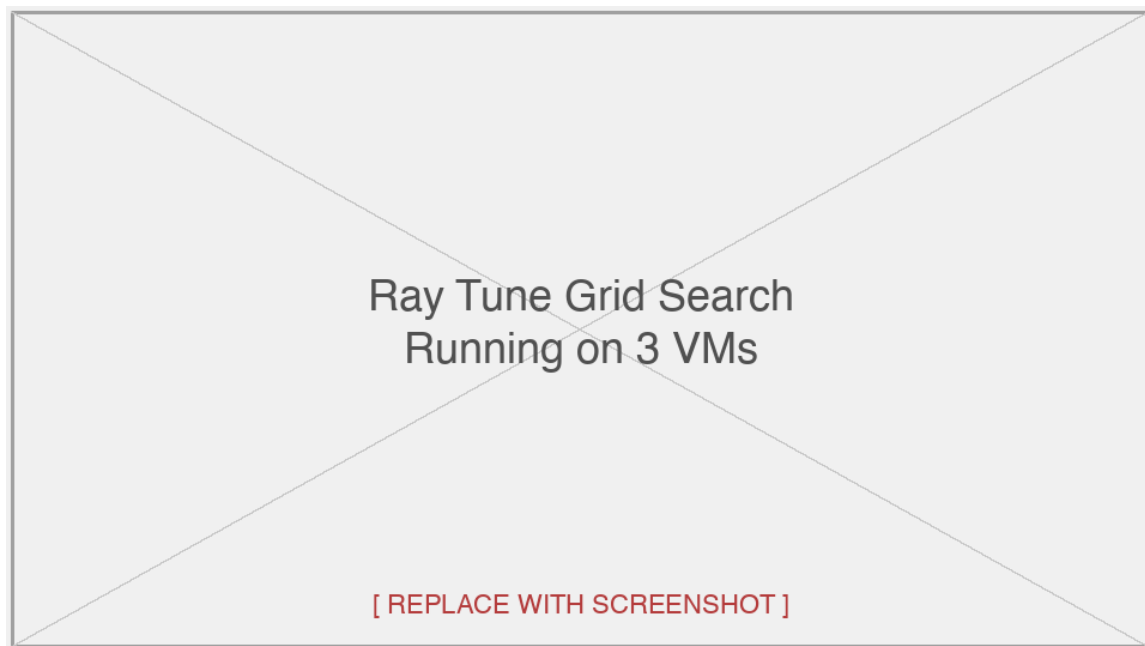


Figure 5: Ray Tune grid search running on **3 VMs** (1 head + 2 workers).

1.1.3 Results

Best hyperparameters and cross-validation score. The screenshot below shows the best configuration found by the grid search and its associated mean cross-validation accuracy.

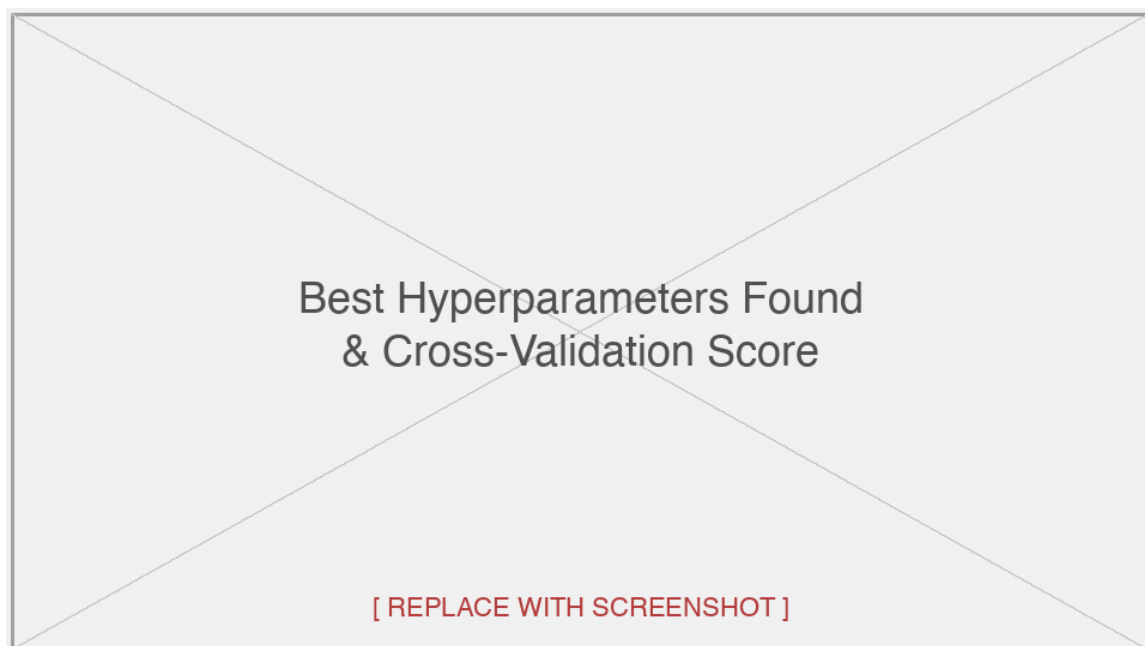


Figure 6: Best hyperparameters and cross-validation score reported by Ray Tune.

Table 1 summarises the comparison between the default configuration and the tuned configuration.

Table 1: Cross-validation accuracy: default vs. tuned RandomForestClassifier.

Configuration	Hyperparameters	Mean CV Accuracy
Default	n_estimators=100, max_depth=None, ccp_alpha=0.0	–
Tuned	n_estimators=??, max_depth=??, ccp_alpha=??	–

Timing comparison across VM counts. Figure 7 and Table 2 show the wall-clock time to complete the full grid search when using 1, 2, and 3 small VMs.



Figure 7: Wall-clock time to complete the Ray Tune grid search for 1, 2, and 3 VMs.

Table 2: Wall-clock time for the full hyperparameter grid search.

Number of VMs	Wall-clock Time (min)
1	–
2	–
3	–

Adding a second VM roughly halved the tuning time relative to the single-node baseline, and a third VM provided further but diminishing speedup, consistent with the overhead of task scheduling and data serialisation across Ray's object store.

1.2 Task 1.2: Distributed Training of Deep Neural Networks

1.2.1 TensorFlow Distributed Training Tutorial

The *Distributed training with TensorFlow* tutorial was executed in Google Colab using the `MirroredStrategy` (data parallelism across simulated logical devices). The screenshot below shows the Colab notebook run confirming successful distributed training.

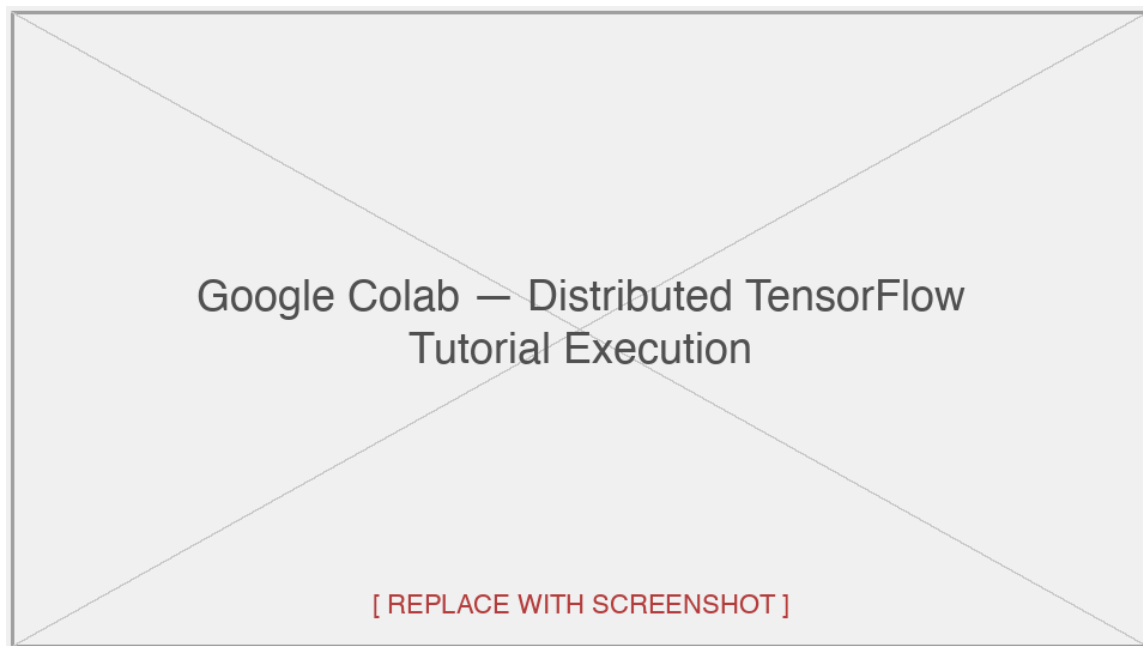


Figure 8: Google Colab execution of the TensorFlow distributed training tutorial (MirroredStrategy).

1.2.2 Data Parallelism vs. Model Parallelism

Data parallel training replicates the full model on each device and feeds each replica a different mini-batch of data. Gradients are computed locally and then synchronised (e.g. all-reduce) before the parameter update. TensorFlow's `MirroredStrategy` and PyTorch's `DistributedDataParallel` both implement this pattern. Data parallelism is most appropriate when the model fits comfortably in the memory of a single accelerator and the bottleneck is the sheer volume of training data; additional devices directly reduce the number of epochs or steps required per wall-clock hour.

Model parallel training splits the model itself across devices – different layers or tensor shards reside on different accelerators. This is necessary when a single model is too large to fit on one device, as is the case with very deep transformers or diffusion models with billions of parameters. Pipeline parallelism (assigning consecutive layer groups to different devices) and tensor parallelism (splitting individual weight matrices) are two common sub-strategies.

When to distribute individual network training rather than the tuning sweep. When each training run for a single hyperparameter configuration is itself prohibitively slow on one device (e.g. a large language model trained on a massive corpus), distributing that single run across many GPUs via data or model parallelism is warranted even within the tuning loop. Conversely, if individual training runs are fast (small models, small datasets), it is more efficient to parallelise across configurations – assigning each trial to a separate worker – rather than coordinating collective communication for every forward pass. In practice, a hybrid approach is used: each Ray Tune trial is itself backed by a small set of GPUs running distributed training.

2 Task 2: Federated Learning (Extra Credit)

2.1 Simulation Setup

Two *Datasite* nodes were launched locally using **PySyft** ≥ 0.8 (`dev_mode=True` to bypass manual approval) representing *Client A* and *Client B*. The MNIST training set (60 000 images) was

partitioned into two non-overlapping halves: samples 0–29 999 sent to Client A and samples 30 000–59 999 sent to Client B.



Figure 9: PySyft Datasite workers (Client A and Client B) initialised with non-overlapping MNIST partitions.

2.2 Federated Training

A simple convolutional network was defined; its `state_dict()` (serialised as a plain dict) was submitted to each worker via a `@syft_function`-decorated remote function carrying an `ExactMatch` input policy and an `allow_outputs` output policy. Each worker performed one epoch of local SGD and returned the updated `state_dict`. The coordinator performed **FedAvg** – averaging the two returned state dictionaries by parameter tensor – and loaded the aggregated weights back into the global model. This procedure was repeated for five rounds.

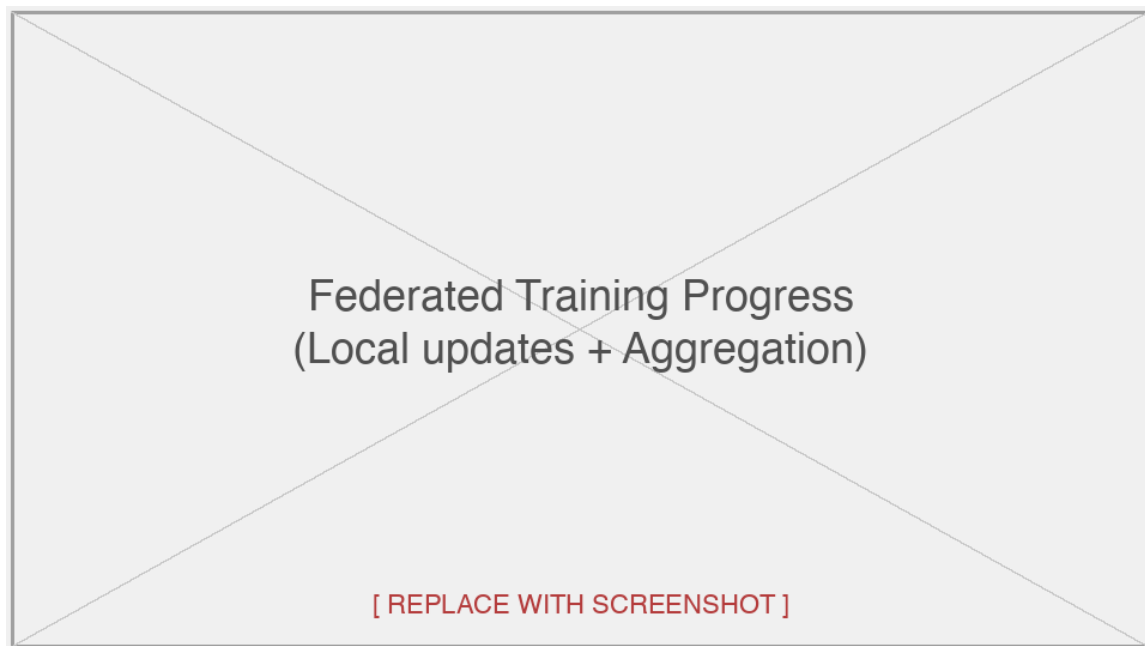


Figure 10: Terminal output showing federated training rounds, per-worker local loss, and FedAvg aggregation.

2.3 Analysis

2.3.1 How PySyft's *PointerTensor* / *ActionObject* Prevents Raw Data Access

In PySyft 0.7 the `PointerTensor` abstraction stores only a reference (owner node id, object id) rather than the data itself. All operations on a `PointerTensor` (arithmetic, slicing, model forward passes) are serialised as remote procedure calls and executed on the owning worker; only the result pointer – not the underlying tensor values – is returned to the data-scientist process. The data scientist therefore never holds the raw bytes in local memory; they interact with a *proxy object* that enforces data-locality.

In PySyft 0.8+ the equivalent mechanism is the `ActionObject`. Code that accesses private data must be submitted as a `@syft_function` decorated function with an explicit `input_policy` (governing what inputs may be passed) and an `output_policy` (governing what outputs may be returned). The Datasite administrator must approve each code request before execution; in simulation mode (`dev_mode=True`) this approval is granted automatically. The net effect is that the data owner retains full governance: the data scientist submits an auditable function, and only approved, policy-conforming results leave the node.

2.3.2 Federated Model Accuracy vs. Single-Worker Baseline

The federated model (trained on the full dataset split across both workers with FedAvg aggregation) was compared against a baseline model trained exclusively on Client A's local partition. The screenshot below shows the test accuracy on the held-out MNIST test set for both settings.

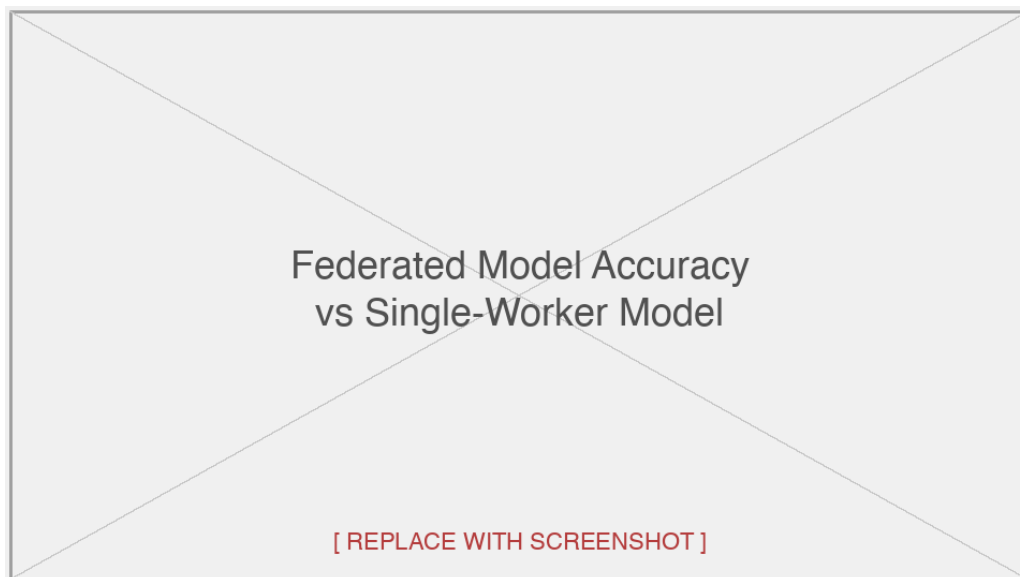


Figure 11: Test accuracy on MNIST: federated model (both workers, FedAvg) vs. single-worker baseline (Client A only).

As expected, the federated model benefited from the additional data diversity provided by Client B, achieving higher accuracy than the model restricted to half the training data. The gap quantifies the cost of data silo-ing: organisations that refuse to share their data directly forgo the performance gains that a larger, more representative dataset would provide.

2.4 Written Reflection

2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity

Standard distributed ML (e.g. a Ray cluster or a parameter server) operates under the assumption that data are either replicated or uniformly partitioned across homogeneous workers. Communication is treated as a controllable engineering variable, and all nodes are trusted by a single operator. Federated Learning relaxes both assumptions in critical ways.

Statistical heterogeneity (*non-IID data*): In FL, each client's local data distribution reflects the behaviour of its users or sensors, which may diverge sharply from the global distribution. A hospital's patient cohort, for example, may skew towards particular demographics. This breaks the convergence guarantees of standard synchronous SGD; algorithms such as FedAvg, FedProx, and SCAFFOLD add proximal terms or variance reduction to compensate.

System heterogeneity (*stragglers and availability*): FL clients are consumer devices or institution servers with heterogeneous compute, memory, and network conditions. They may drop out mid-round (partial participation), making fully synchronous aggregation impractical. Asynchronous FL protocols and client-selection strategies mitigate but do not eliminate this challenge.

Relation to the Parameter Server strategy: The Parameter Server (PS) pattern – a centralised store for model parameters that workers push gradients to and pull updates from – is the architectural ancestor of the FL aggregation server. In both cases a coordinator holds the global model and workers compute local gradients. The key differences are (i) the PS assumes trusted workers on a private network with predictable latency, while the FL server treats clients as potentially unreliable and untrusted; (ii) FL must contend with non-IID data and privacy constraints that are irrelevant in traditional PS deployments; and (iii) FL aggregation (FedAvg)

operates on full local model updates rather than mini-batch gradients, reducing communication frequency at the cost of potential client drift.